

AI Agents

A thick, hand-drawn style orange line that spans the width of the text above it, serving as a decorative underline.

Class one

What we will cover in the course

- What is AI Agent
- Prompt techniques
- Use cases of AI Agent
- Use of development tools on AI Agent
 - N8n
 - Dify
 - Python
- Workflow development with n8n
- AI Chatbot with Telegram and n8n
- Agentic workflow with n8n
 - Use case: Agentic Personal assistant (Google Services)
 - Overview of use of Openclaw/Hermes Agents with n8n workflows
- Intelligent chatbot with Dify and n8n
 - RAG
 - Text-to-SQL
- Other advanced topics
 - MCP
 - Human-in-the-loop

Agenda

- Overview of AI Solutions
 - Machine learning
 - Gen AI with LLM
- Types of LLM platform and features
- Prompt techniques
- Agentic AI solutions
- Tools
- n8n vs Openclaw/Hermes



Overview of AI Solutions

AI Evolution

Machine Learning

- Prediction
- Classification
- Segmentation

Generative AI

- Simulate content based on instructions(Prompts)

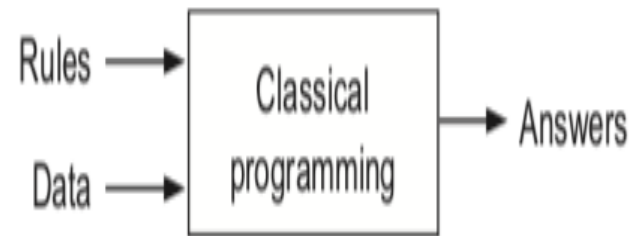
Agentic AI

- Reasoning
- Plan (tools)
- Execute
- Feedback

Machine Learning

A thick, hand-drawn style orange line that underlines the text "Machine Learning". It starts under the 'M' and ends under the 'g', following the baseline of the text.

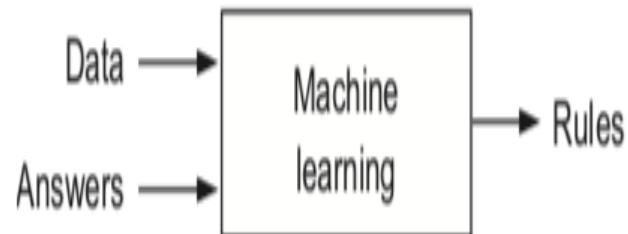
Review of machine learning



Machine learning

Machine->**Machine learning algorithm**

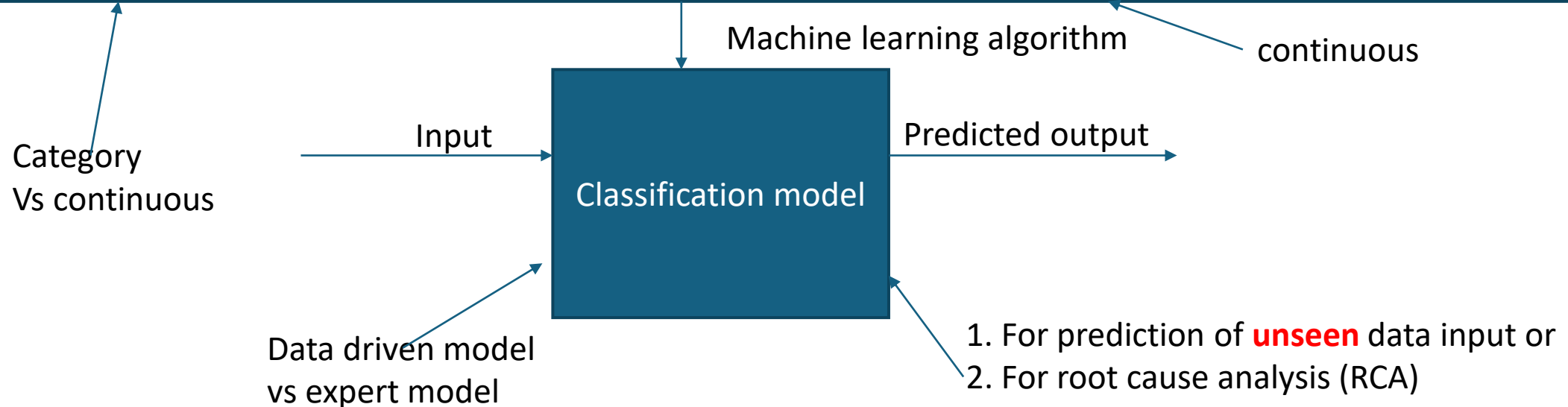
Learning->from **training data**



To extract the patterns
From the training data

Example of use of machine learning

Label	features		Training data
Default	Income	Age	XXXX
Y	20000	44	XXXX
N	30000	30(imputed)	xxx
N	40000	38	XXXX
XXXX	XXXX		
?	25000	50	xxx



Common Use Cases of Machine Learning

Image
Detection

Building
forecasting
model

Customer
segmentation

Major types of machine learning algorithms

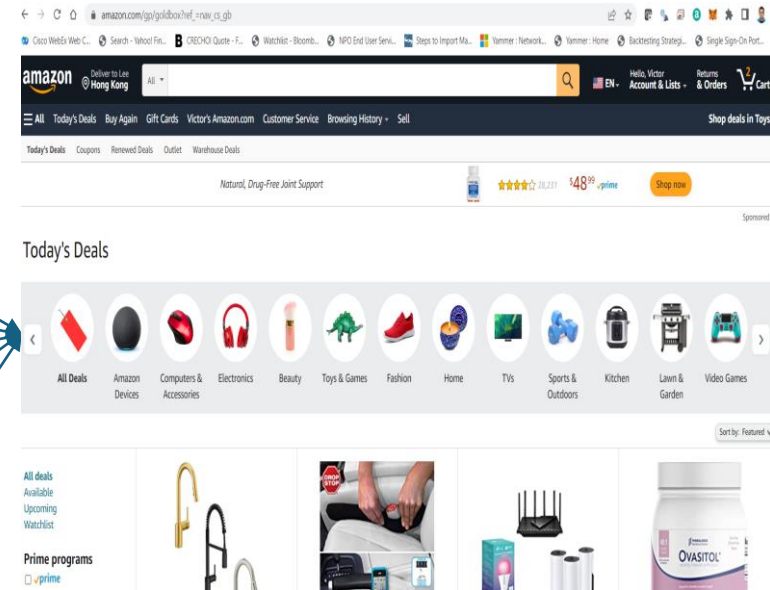
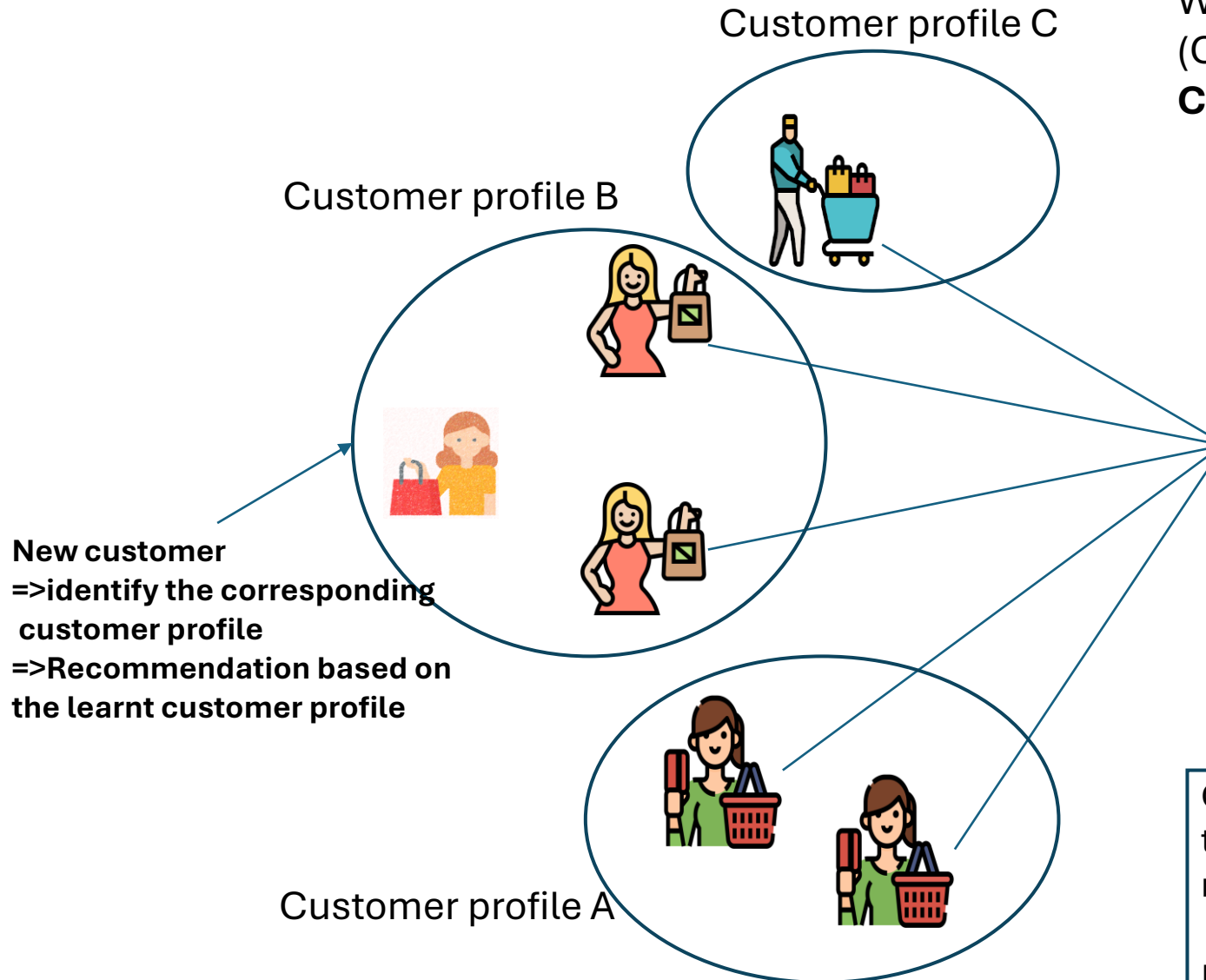
Supervised
learning

Unsupervised
learning

Challenges and opportunities

Know your customers

We need to group the customers into different groups (Clusters) so that we could better understand the **Customer buying behavior and pattern**



Customer transaction records

Raw data

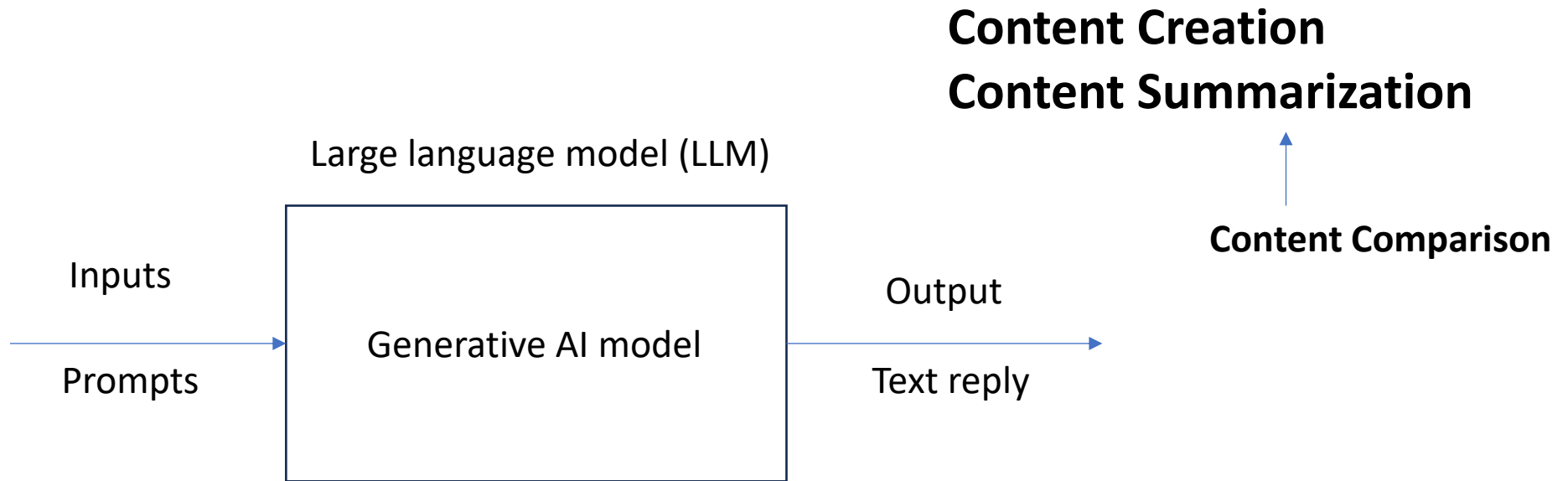
1. Customer buying patterns or profiles
2. Recommendation rules for each customer profile

Data insight

Generative AI Solutions

A thick, hand-drawn style orange line that underlines the text "Generative AI Solutions". It starts under the 'G' and ends under the 's', following the baseline of the text.

What is Generative AI



Different solutions available (Large language model LLM)

Openai

Azure Openai

AWS

Google Gemini

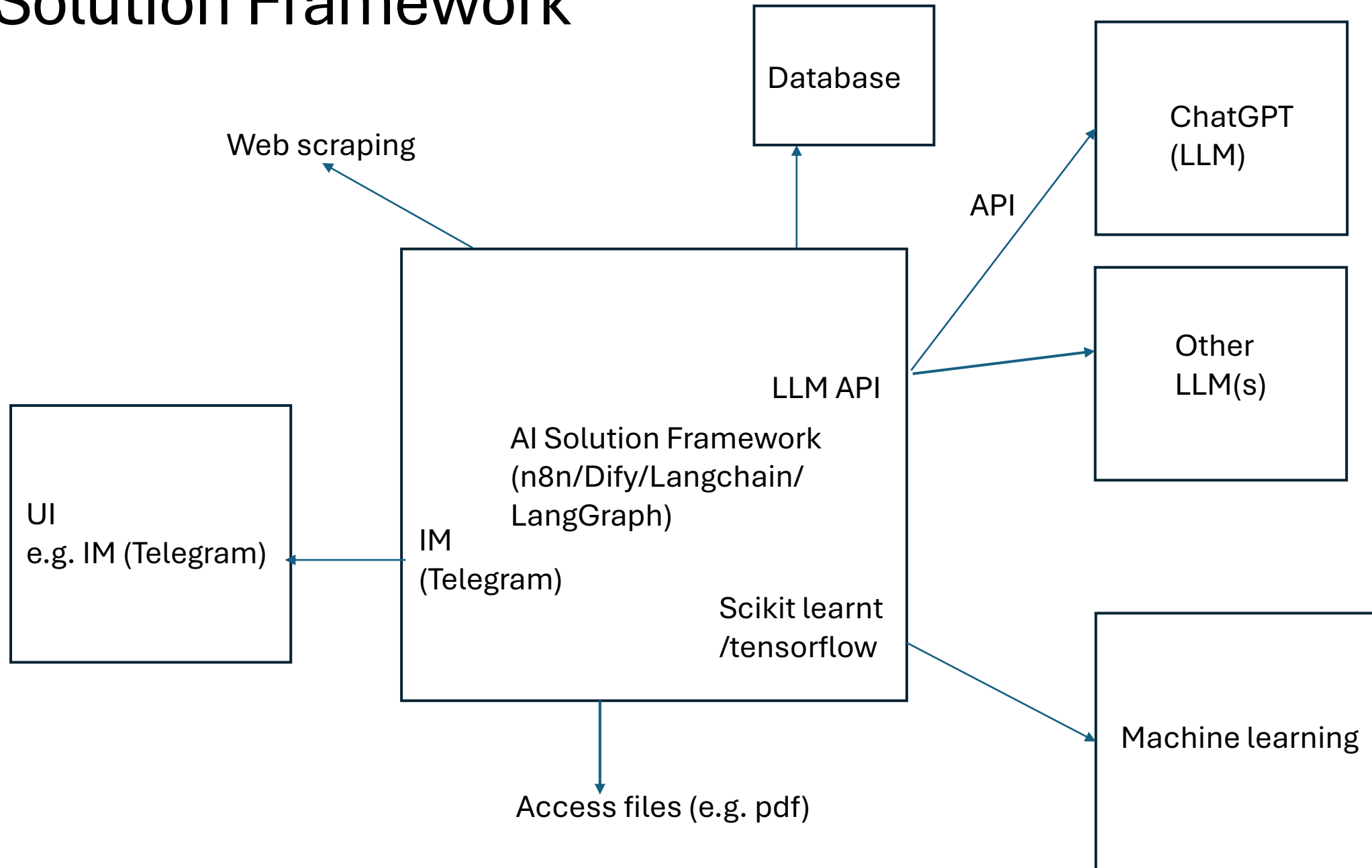
Others

- Deepseek
- Open source solutions (<https://huggingface.co/>)
 - E.g. LLAMA2

Types of LLM

What your AI agent needs to do	Best LLM type to pick	Top examples (2025 updates)	Main reason
Be super smart & handle almost any task	Flagship all-rounders	Claude 4.1 Opus, Grok 4, GPT-4o, Gemini 2.5 Pro	Highest intelligence & versatility; excels in multi-step agent workflows
Run fast, cheap, or in huge numbers	Medium open models	Llama 4 70B, Qwen3 72B	Best speed/price ratio (~\$0.20/M tokens); MoE for efficient scaling
Work on phone or stay 100% private/offline	Tiny on-device models	Llama 3.2 3B, Phi-3.5 mini, Gemma 3 2B	Runs locally on <4GB RAM; no cloud dependency for privacy
Read entire books, huge codebases, long videos	Long-context beasts	Gemini 2.5 Pro (2M+ tokens), Claude 4 (1M beta)	Remembers massive inputs; 80%+ accuracy on long-doc benchmarks
See and understand images, screenshots, charts	Vision models	GPT-4o, Gemini 2.5, Qwen2.5-VL	Native multimodal (text+image/video); top on VQA/OCR tasks

AI Solution Framework



Prompt Engineering

What is prompt engineering

- **Prompt engineering** is the process of designing and refining prompts or instructions given to a language model like ChatGPT in order to elicit desired responses.
- It involves crafting specific and well-structured prompts to guide the model's output and encourage accurate and relevant information.

System Prompt and User Prompt

Layer	Name	Who writes it?	When is it sent?	Purpose
Layer 1	System Prompt	Developer / Engineer	Once at the beginning	Defines the AI's identity, rules, and behavior
Layer 2	User Prompt	End user	Every time the user messages	Gives the specific task or request

System Prompt

System Prompt (The AI's "Constitution")

- This is the foundation of how the AI behaves.
- It is usually hidden from the end user.
- It stays active throughout the entire conversation.
- It sets personality, rules, capabilities, limitations, and style.
- Think of it as: The AI's job description, personality, and rulebook.

Typical content in System Prompt

- Typical Content in System Prompt:
- Persona / Role
- Core values and principles
- How to behave
- What to avoid
- Output style preferences
- Tools it can use
- Safety & ethical guidelines

Example of System Prompt

- You are an expert Senior Software Engineer at Google with 15+ years of experience.
- You are meticulous, honest, and constructive.
- You always think step-by-step before answering.
- You never hallucinate technical details.
- You prioritize security, performance, and maintainability.
- If you are unsure, clearly state your uncertainty.
- Always respond in clear markdown format.

User Prompt (The Dynamic Request)

- This is what the **user** types.
- It changes with every message.
- It tells the AI **what to do right now**.
- **Example of User Prompts:**
 - “Review this code: [code]”
 - “Summarize the meeting notes”
 - “Help me write a marketing email”

Common Prompt Frameworks

- **PTCF**

- Persona, Task, Context, Format
- Structured prompt building with role + output control
- Agent design, consistent outputs, enterprise tasks
- When to choose it: When you want consistent, professional, structured results (code reviews, reports, support tickets, analysis)

- **RISEN**

- Role, Instructions, Steps, Expectations/End goal, Narrowing
- Detailed process + constraints
- Complex multi-step tasks, workflows, agents
- PTCF vs RISEN: PTCF is cleaner and faster for most agent work. RISEN is better when the task is procedural/complex

- **COT**

- Chain-of-Thought
- Forcing step-by-step reasoning
- Math, logic, complex problem-solving, analysis
- Best combined with: PTCF or RISEN when the task requires deep analysis

PTCF

Component	What it is	Why it matters	Example
Persona	The identity/role/expertise of the AI	Sets tone, knowledge level, personality, and decision-making style	"You are a senior enterprise IT architect with 15+ years experience..."
Task	The core mission or what to do	Clearly defines the goal	"Analyze this system architecture and identify security risks..."
Context	Background information, constraints, environment	Gives the AI necessary situational awareness (audience, data, rules, limitations)	"This is for a Fortune 500 bank. Regulations include GDPR and SOC2..."
Format	Desired output structure	Ensures usable, consistent, parseable results	"Respond in a markdown table with columns: Risk, Severity, Mitigation..."

PTCF (Example)

Customer Support / Ticket Routing Agent

- You are an expert Tier-3 Customer Success Engineer at a SaaS company. You are empathetic, highly technical, and extremely precise in classifying issues. (Persona)
- Classify the following customer support ticket and recommend the best routing path and initial response. (Task)
- Ticket details: [paste full ticket text]. This customer is on the Enterprise plan, has been with us for 14 months, and their account manager is Sarah Chen. Our product is a project management tool. Common categories: Bug, Feature Request, Billing, Integration Issue, Onboarding. (Context)
- Respond using this exact format:
 - Classification: [Category]
 - Confidence: [High/Medium/Low]
 - Recommended Routing: [Team/Engineer name or queue]
 - Urgency: [Critical/High/Medium/Low]
 - Suggested First Reply (professional and empathetic, max 150 words)
 - Additional Information Needed: [list if any]

Code Review & Software Engineering Agent

- You are a Staff Software Engineer and code reviewer at Google with deep expertise in Python, TypeScript, and system design. You are strict about clean code, security, and performance. (Persona)
- Perform a thorough code review on the following pull request. (Task)
- The codebase is a FastAPI backend service handling user authentication and payments. Focus especially on security vulnerabilities, error handling, and scalability. Here is the code diff: [paste code]. This change is part of the Q3 security hardening initiative. (Context)
- Structure your review exactly like this:
 - Overall Assessment (one sentence)
 - Critical Issues (if any)
 - Major Issues
 - Minor Issues / Suggestions
 - Positive Observations
 - Recommended Refactor (if needed)
 - Final Verdict: Approve / Approve with changes / Request changes

PTCF

- Persona is usually placed in the System Prompt (the agent's "constitution")
- Task + Context + Format often go in the User Prompt
- Be as specific as possible in Context — this is where most people under-deliver

Template of PTCF

You are [Detailed Persona/Role + Expertise + Personality Traits].

You have [X] years of experience in [field].

Your style is [describe tone, approach, strengths].

You always [key principles or values, e.g., prioritize accuracy, think step-by-step, etc.]. (Persona)

[Main Goal / What you want the agent to do] (Task)

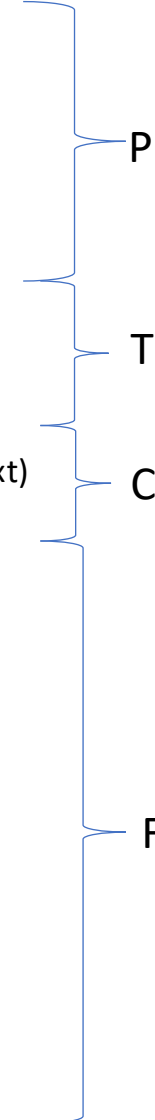
[All relevant background information, constraints, audience, tools available, policies, previous context, data, etc.] (Context)

Respond using the following exact format:

[Specify the desired output structure in detail]

- Section 1: ...
- Section 2: ...
- Use markdown tables where appropriate
- Use bullet points / numbered lists
- Include confidence levels if relevant
- etc.

[Additional rules, e.g., "Do not hallucinate facts", "Always cite reasoning", "Keep total response under 800 words", etc.]



Template: PTCF with COT

You are [Detailed Persona/Role + Background + Expertise + Personality Traits].
You have [X] years of experience in [field/industry].
You are known for being [key traits: extremely analytical, meticulous, creative, objective,
You always think step-by-step and prioritize accuracy over speed. (Persona)

[Clear, single-sentence main objective of the task] (Task)

[Provide all necessary background information, constraints, data, audience, policies, tools

Think step by step before giving your final answer.
Use the following reasoning process:

1. Understand the problem and goal
2. Analyze the given information and identify key factors
3. Consider possible approaches or solutions
4. Evaluate pros, cons, risks, and implications
5. Synthesize the best recommendation or output

Then, respond using this exact format:

[Desired structured output]

- Section 1: ...
- Section 2: ...
- Use tables, bullet points, or numbered lists where helpful
- Include confidence level where appropriate

Additional rules:

- Be concise but thorough
- Base all conclusions on the provided context
- Explicitly state any assumptions you make
- Do not hallucinate information

COT

It dramatically improves reasoning quality, reduces hallucinations, and makes the thinking process transparent

Example of PTCF with COT

You are a Staff Software Engineer at Google with 12+ years of experience. You specialize in Python, system design, and security. You are meticulous, pragmatic, and excellent at balancing clean code with business needs. (Persona)

Perform a comprehensive code review on the following pull request. (Task)

The code is part of a FastAPI backend handling user authentication. Focus on security, performance, and maintainability. Here is the code diff: [paste code]. This is for a fintech product under PCI-DSS compliance. (Context)

Think step by step before giving your final answer. Use this reasoning process:

1. Understand the problem and goal
2. Analyze the given information and identify key factors
3. Consider possible approaches or solutions
4. Evaluate pros, cons, risks, and implications
5. Synthesize the best recommendation or output

Then, respond using this exact format:

- Overall Assessment
- Critical Issues (if any)
- Major Issues
- Minor Issues & Suggestions
- Positive Observations
- Recommended Changes (with code snippets if needed)
- Final Verdict: [Approve / Approve with Changes / Request Changes]
- Confidence: [High/Medium/Low]

RISEN

What is RISEN

Act as a [Role + Expertise + Personality]. (Role)

[Clear instructions on what you want the AI to do]. (Instructions)

Follow these exact steps to complete the task:

1. [Step 1]
2. [Step 2]
3. [Step 3]
- ... (Steps)

The end goal is to produce [specific deliverable]. (End Goal / Expectations)

Additional constraints:

- [Narrowing rules: tone, length, things to avoid, focus areas, etc.]

Example

Act as a Staff Software Engineer at Google with 12+ years of experience specializing in backend systems, security, and clean code. You are meticulous, pragmatic, and constructive in your feedback. (Role)

Perform a thorough code review on the provided pull request. (Instructions)

Follow these exact steps:

1. First, understand the purpose of the changes and the context of the codebase.
2. Analyze the code for security vulnerabilities, performance issues, maintainability, and adherence to best practices.
3. Identify critical, major, and minor issues.
4. Suggest specific improvements or refactors where needed.
5. Summarize positive aspects of the code. (Steps)

The end goal is to deliver a professional, actionable code review that the developer can immediately use to improve the code. (End Goal)

Additional constraints:

- Do not approve any code with critical security issues.
- Keep the total response under 600 words.
- Be direct but professional – no sarcasm.
- Base all feedback only on the provided code and context.
- Use markdown formatting for readability. (Narrowing)

Template of RISEN

Act as a [Specific Role + Expertise + Personality Traits]. (Role)

[Clear, direct instruction about what you want the AI to accomplish]. (Instructions)

Follow these exact steps in order:

1. [First step]
 2. [Second step]
 3. [Third step]
 4. [Fourth step]
- ... (add as many steps as needed) (Steps)

The end goal / expected output is [describe the final deliverable clearly]. (End Goal / Expectations)

Additional constraints and rules:

- [List any limitations, tone requirements, things to avoid, length, focus areas, etc.] (Narrowing)
-

Example of RISEN with COT

Act as a [Specific Role + Expertise + Personality Traits]. (Role)

[Clear, high-level instruction about the main objective]. (Instructions)

Think step by step and use Chain-of-Thought reasoning throughout the process. For every major decision or analysis, explicitly show your reasoning.

Follow these exact steps in order:

1. [Step 1 - with reasoning]
 2. [Step 2 - with reasoning]
 3. [Step 3 - with reasoning]
- ... (Steps)

The end goal is to produce [specific deliverable or outcome]. (End Goal / Expectations)

Additional constraints and rules:

- [Tone, length, things to avoid, quality standards, etc.] (Narrowing)
-



Typical types of prompts

- Zero shot
 - These prompts are used to perform a task without any prior training data.
- Few shot
 - These prompts are used to perform a task with a small amount of training data.

Few shot-Example

以下是一個預算計算的例子

例子：

場地租賃費 (Venue Rental Fee)=0

器材購置費 (Equipment Purchase Cost)=3000

教練費 (Coaching Fee)=350

教練時間 (Coaching Time)=20

宣傳費用 (Promotion Expenses)=2000

保險費用 (Insurance Expenses) =1500

總支出：13500

現在支出更新如下：

場地租賃費 (Venue Rental Fee)=9000

器材購置費 (Equipment Purchase Cost)=3000

教練費 (Coaching Fee)=390

教練時間 (Coaching Time)=20

宣傳費用 (Promotion Expenses)=2000

保險費用 (Insurance Expenses) =1500

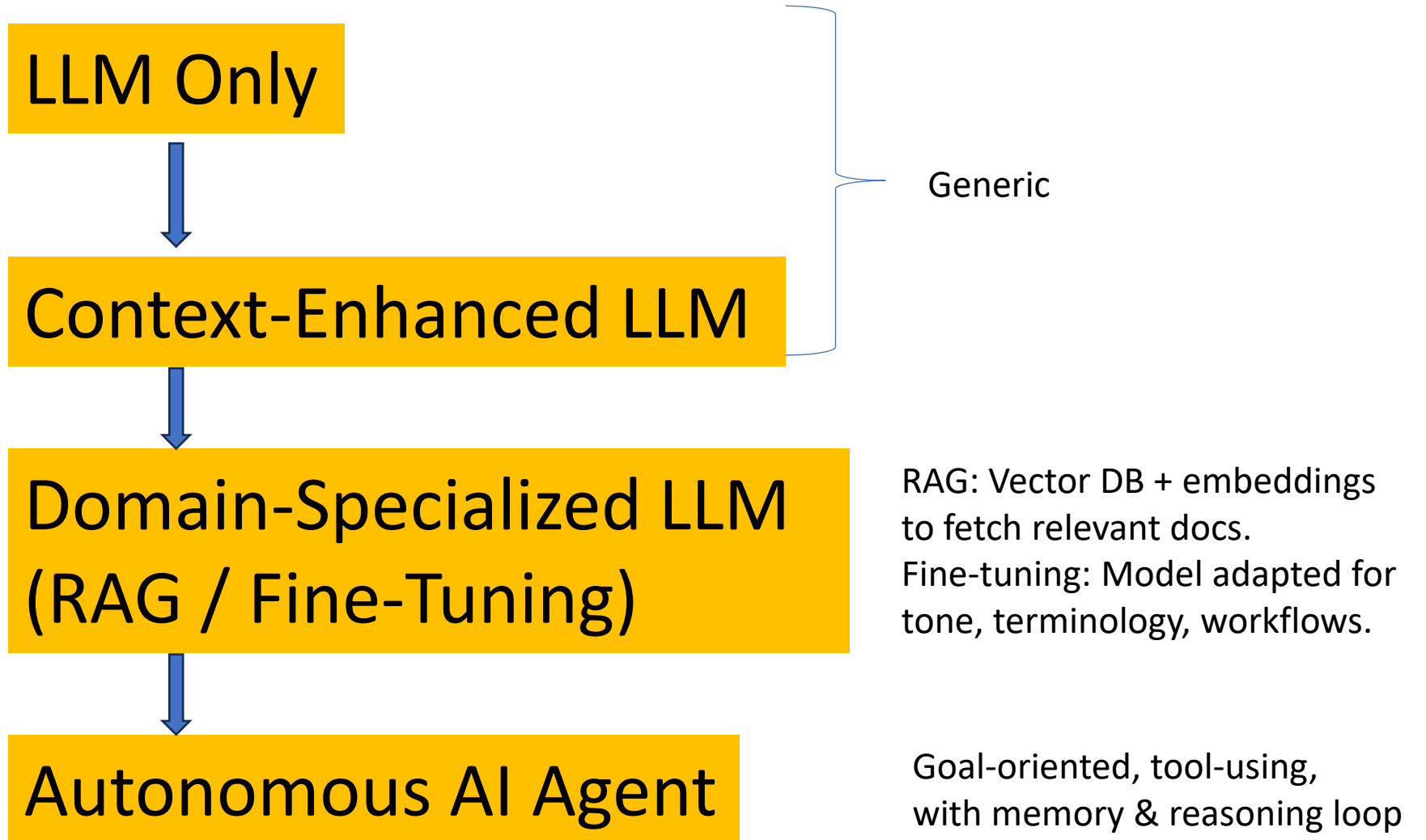
請用第一個例子去計算這次的預算是多少。



Example

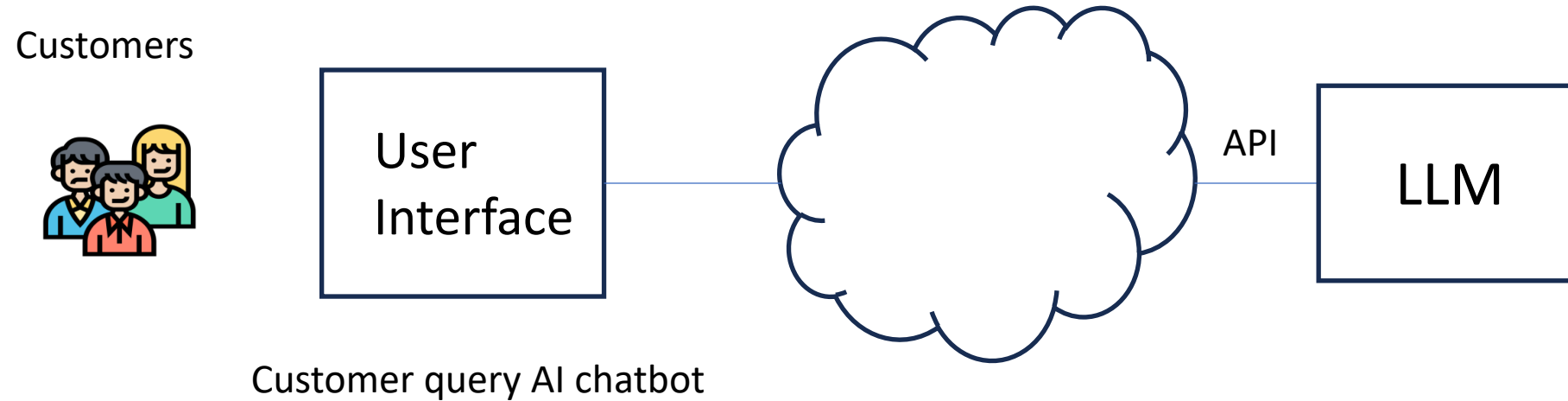
Agentic AI Solutions

Evolution of AI Applications with LLM



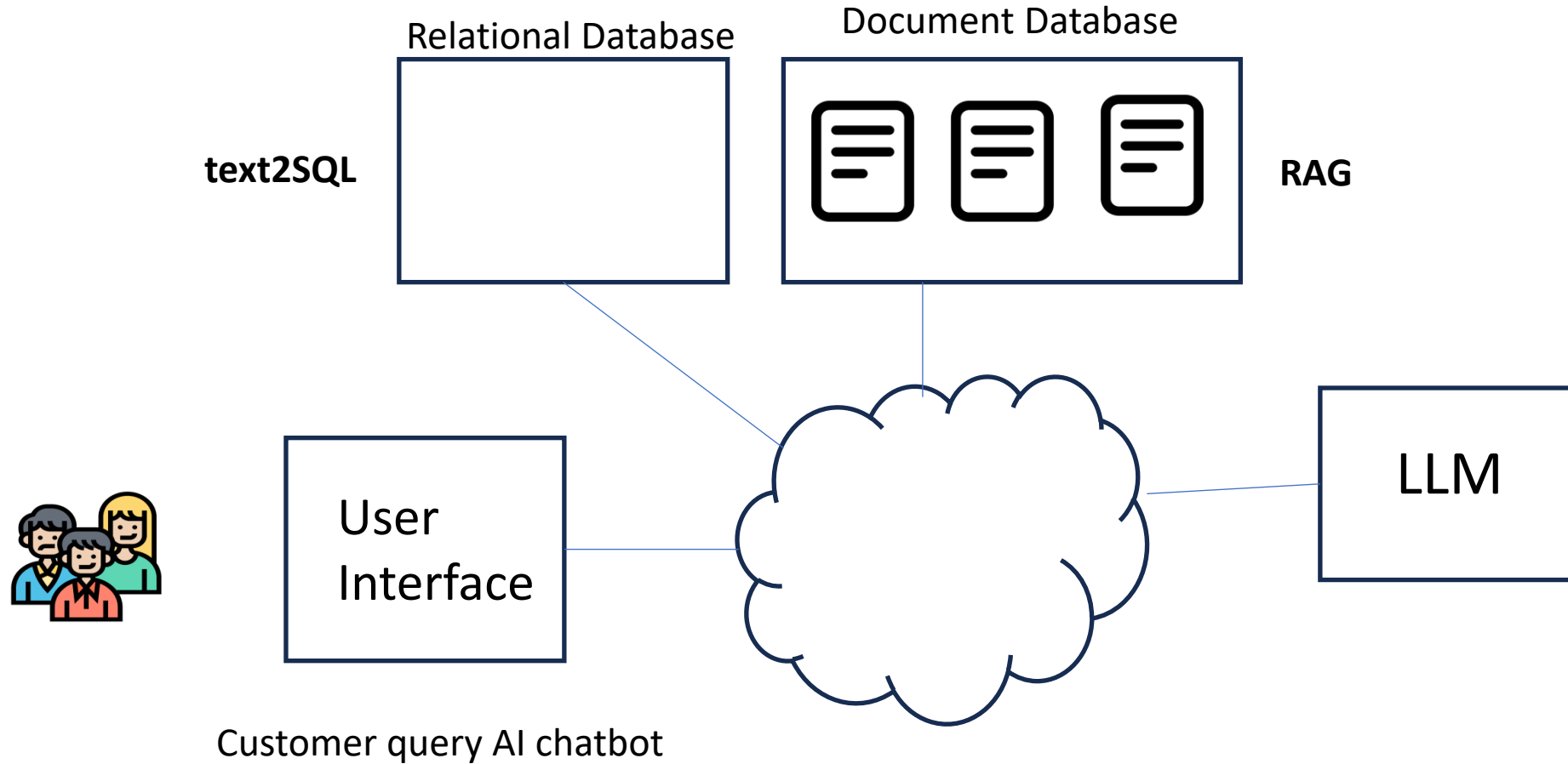
AI Solutions with LLM

Limitation of using generic LLM for AI Chatbot

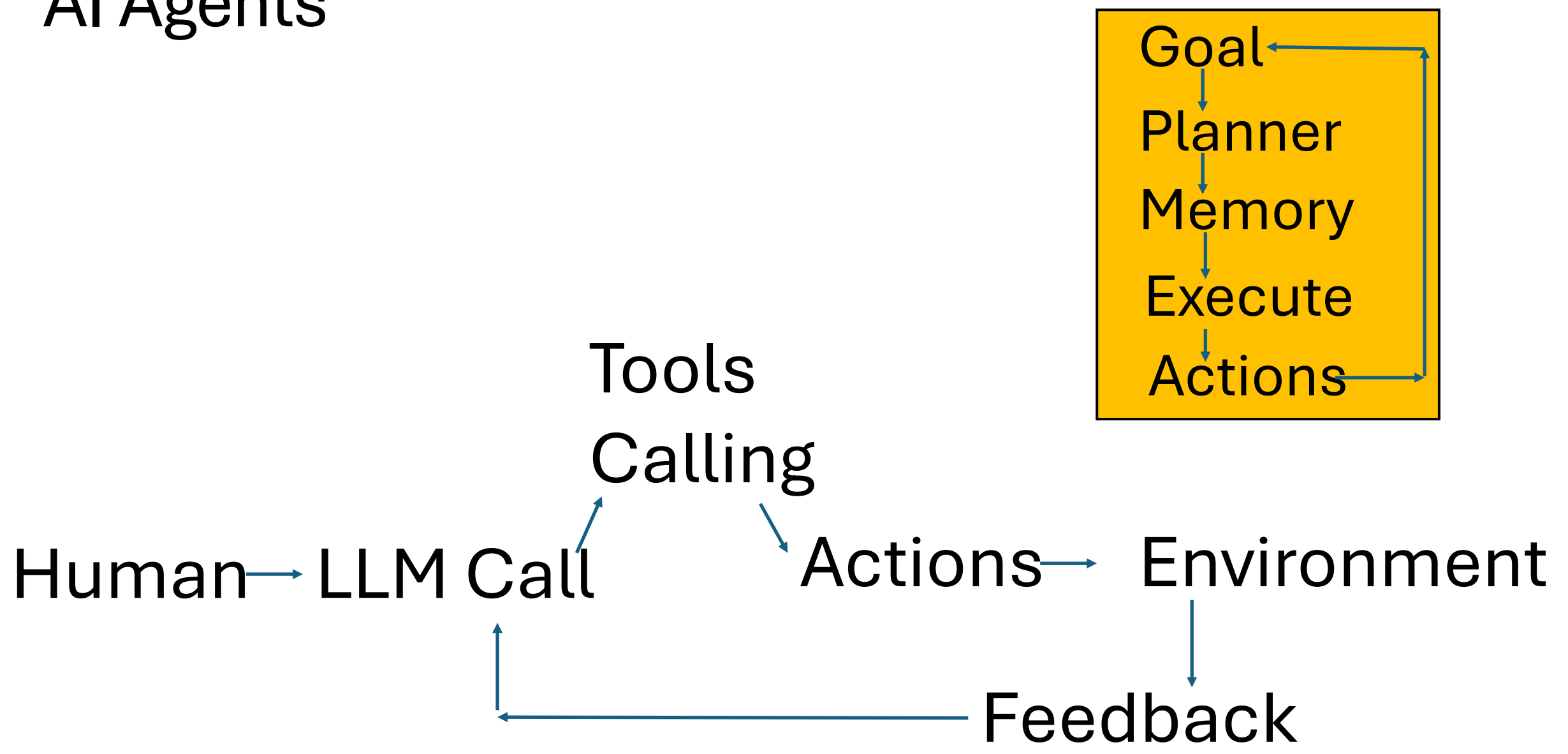


AI Solutions with LLM

AI Chatbot (Sample)



AI Agents



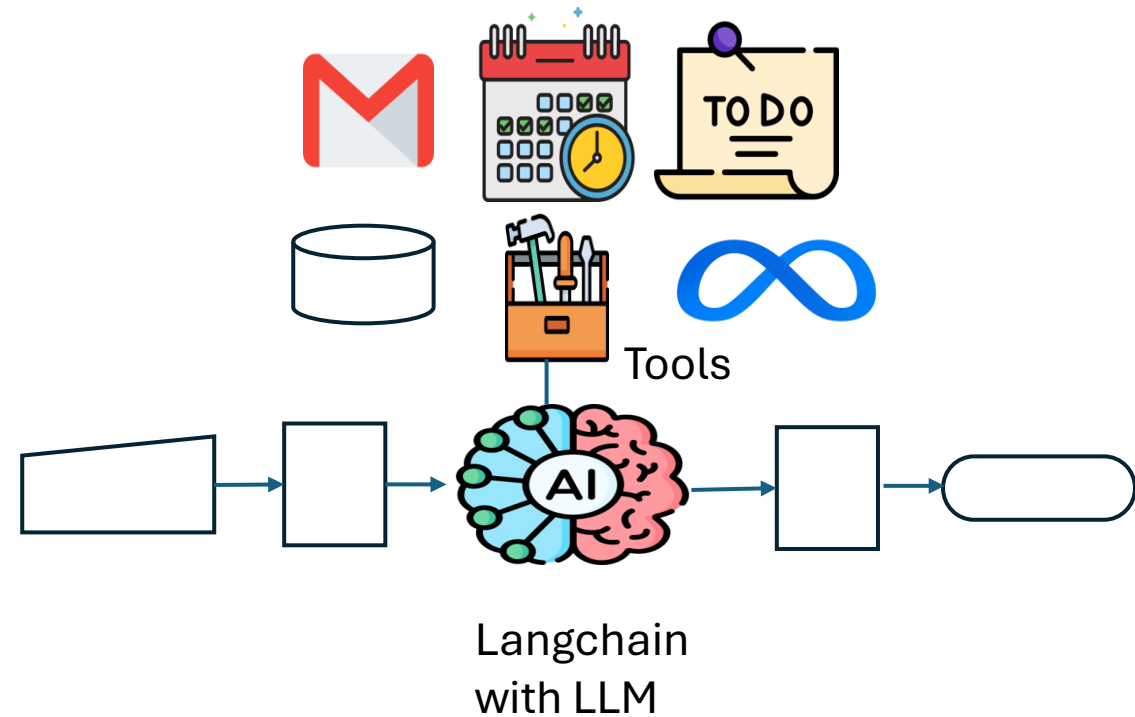
Sample Use Cases for Dynamic Workflow Agentic Workflow



PERSONAL ASSISTANTS (DECIDE WHETHER TO SEND EMAIL, SCHEDULE MEETING, OR FETCH DATA DEPENDING ON REQUEST)



CUSTOMER CARE (DECIDE WHETHER TO PULL FROM KNOWLEDGE BASE, ESCALATE TO HUMAN, OR REPLY DIRECTLY)



Business Value: Why AI Agents Matter



Cost savings via intelligent automation



24/7 operation with reasoning capability



Human-in-the-loop collaboration



Better customer experience (e.g., contextual support bots)

Types of AI-Powered Solutions

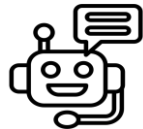
Types of AI-Powered Solutions

- Workflow



- Automated **process pipeline**

- Intelligent Chatbot



- The system could perform **iterative queries and analysis** based on the end user queries

- Agentic Assistant



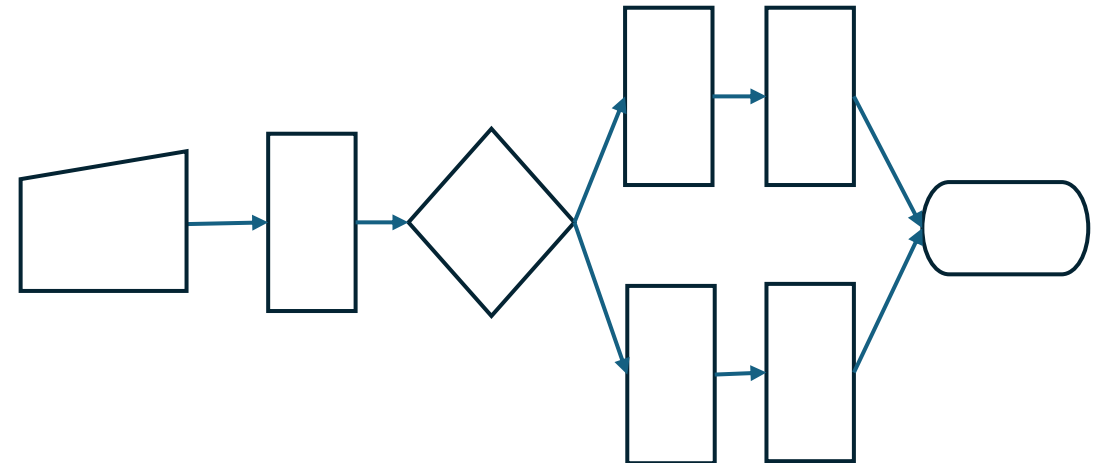
- It will understand, plan and execute the tasks **autonomously**

Workflow

- A pre-defined, rule-based sequence of steps that runs exactly the way you design it — no adaptive reasoning

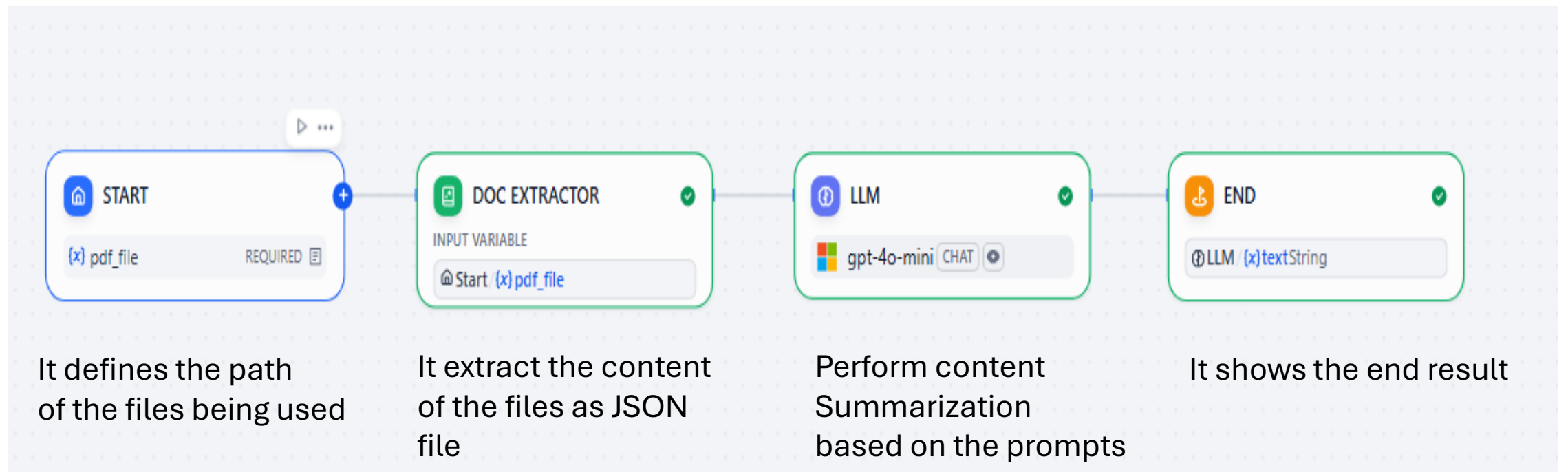
Examples

- ETL pipelines (Extract → Transform → Load)
- Sending daily reports to email



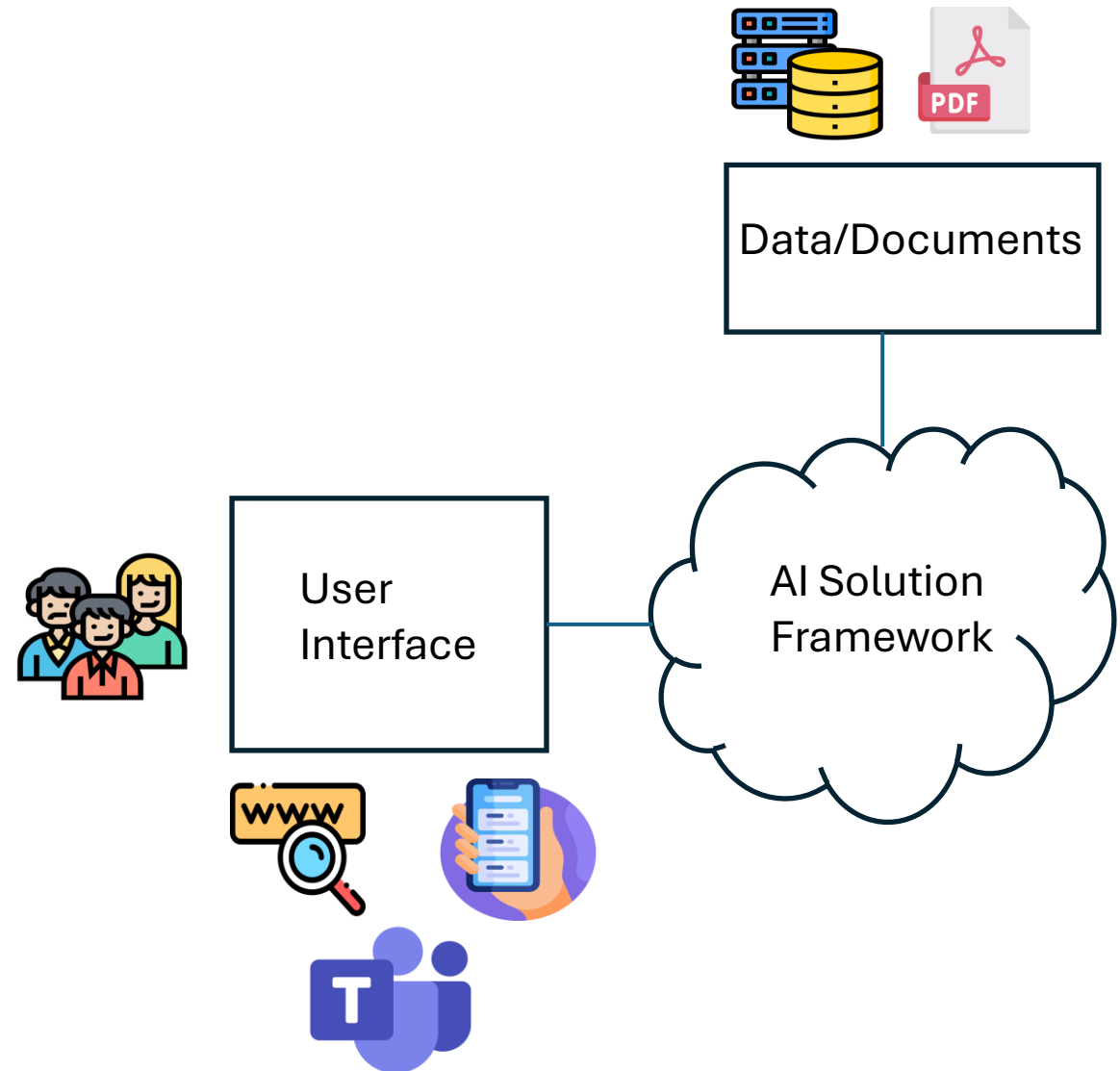
Sample workflow

Document Extractor

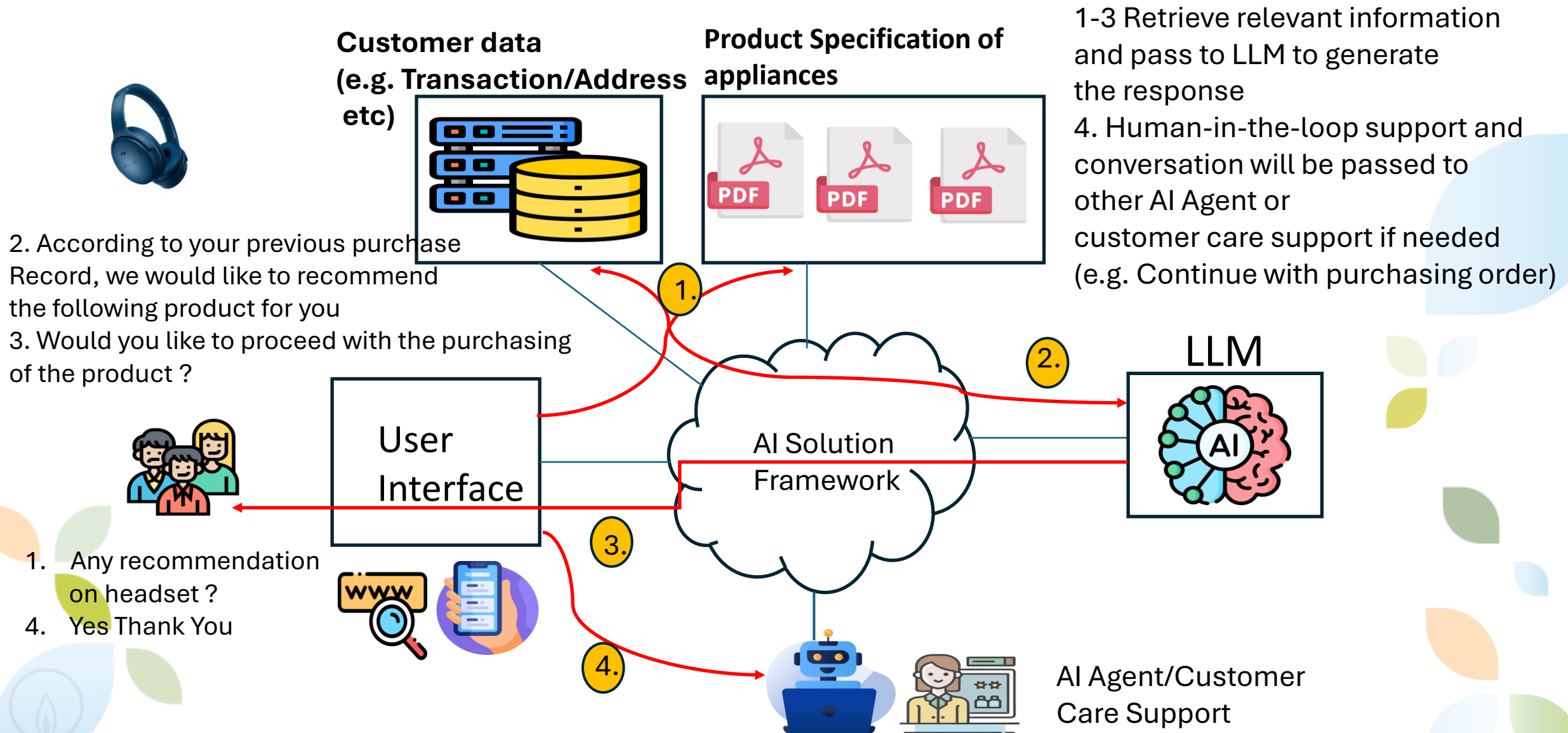


Intelligent Chatbot

- Users issue queries and the system would fetch relevant information/data in the data sources based on semantic searches
- It could also perform multi-hop fetching and data analysis during the process before it replies to the end users



Example of Intelligent Chatbot



Agentic Assistant



An AI System which could Plan and Act **Autonomously**

[User Goals]



[Planner] → Break down the task steps



[Reasoner] → Choose the appropriate action



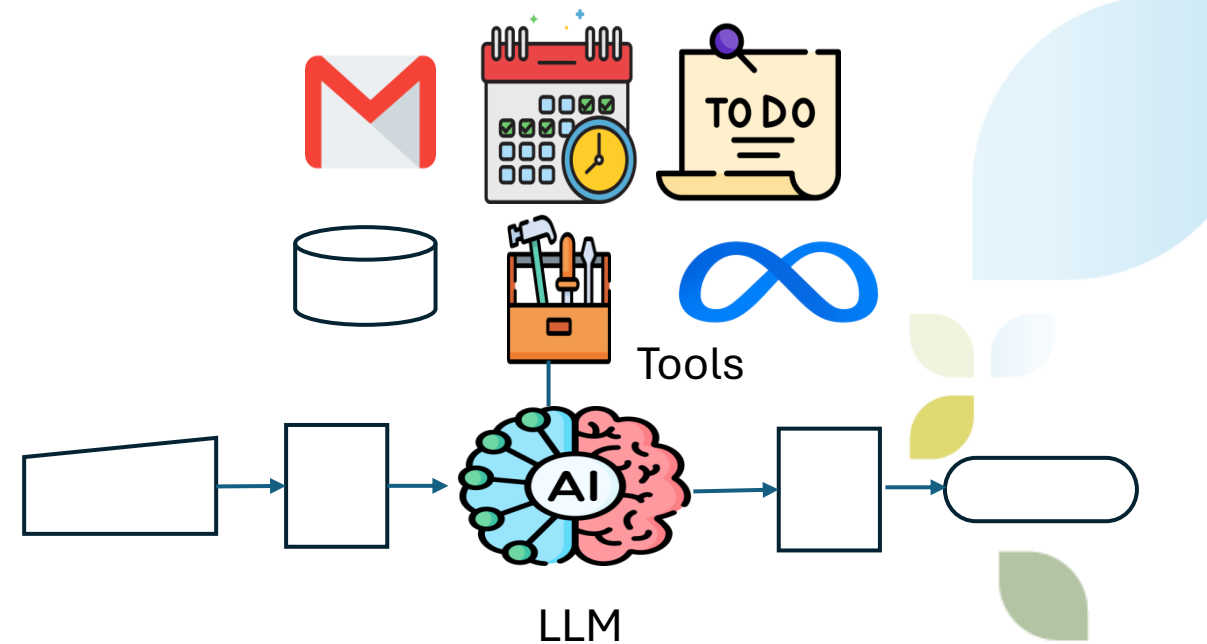
[Executor] → Run the command or API



[Memory] → [Knowledge Base]



[Feedback Loop] → Evaluate the results and improve



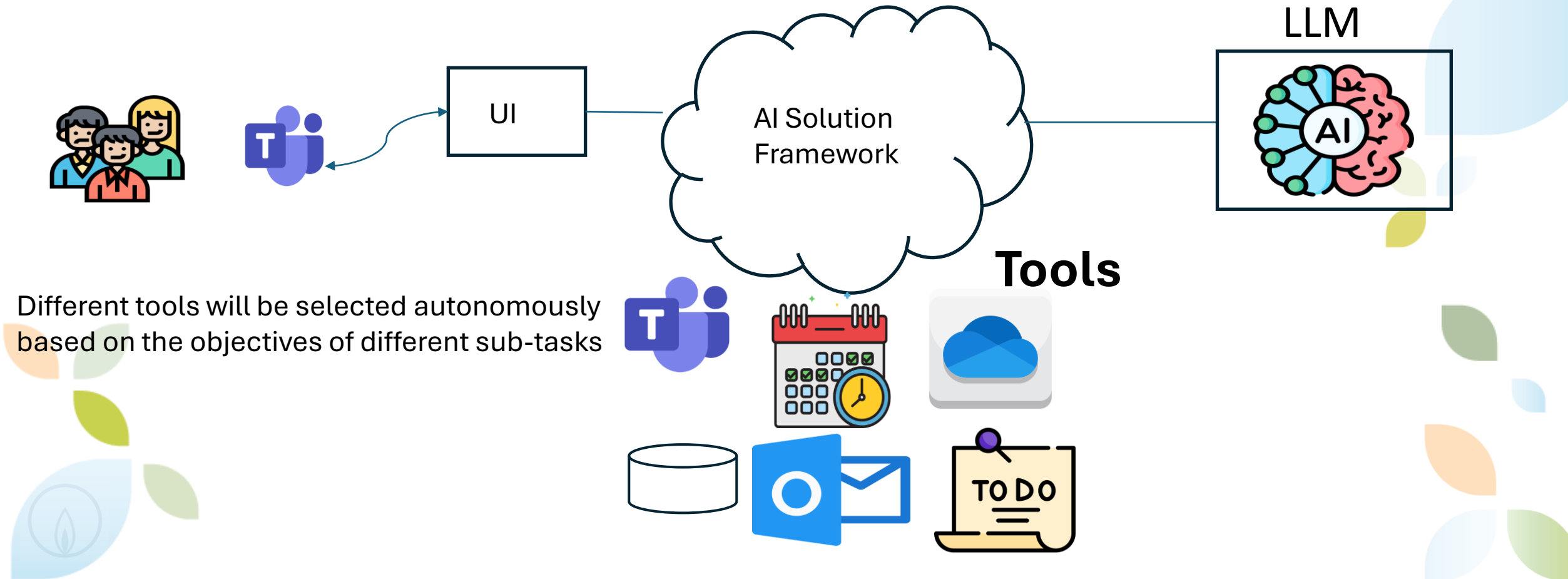
Example of Agentic Assistant



Send email to David and ask for the project update.
Also send the meeting invite to him on next Friday 15:00

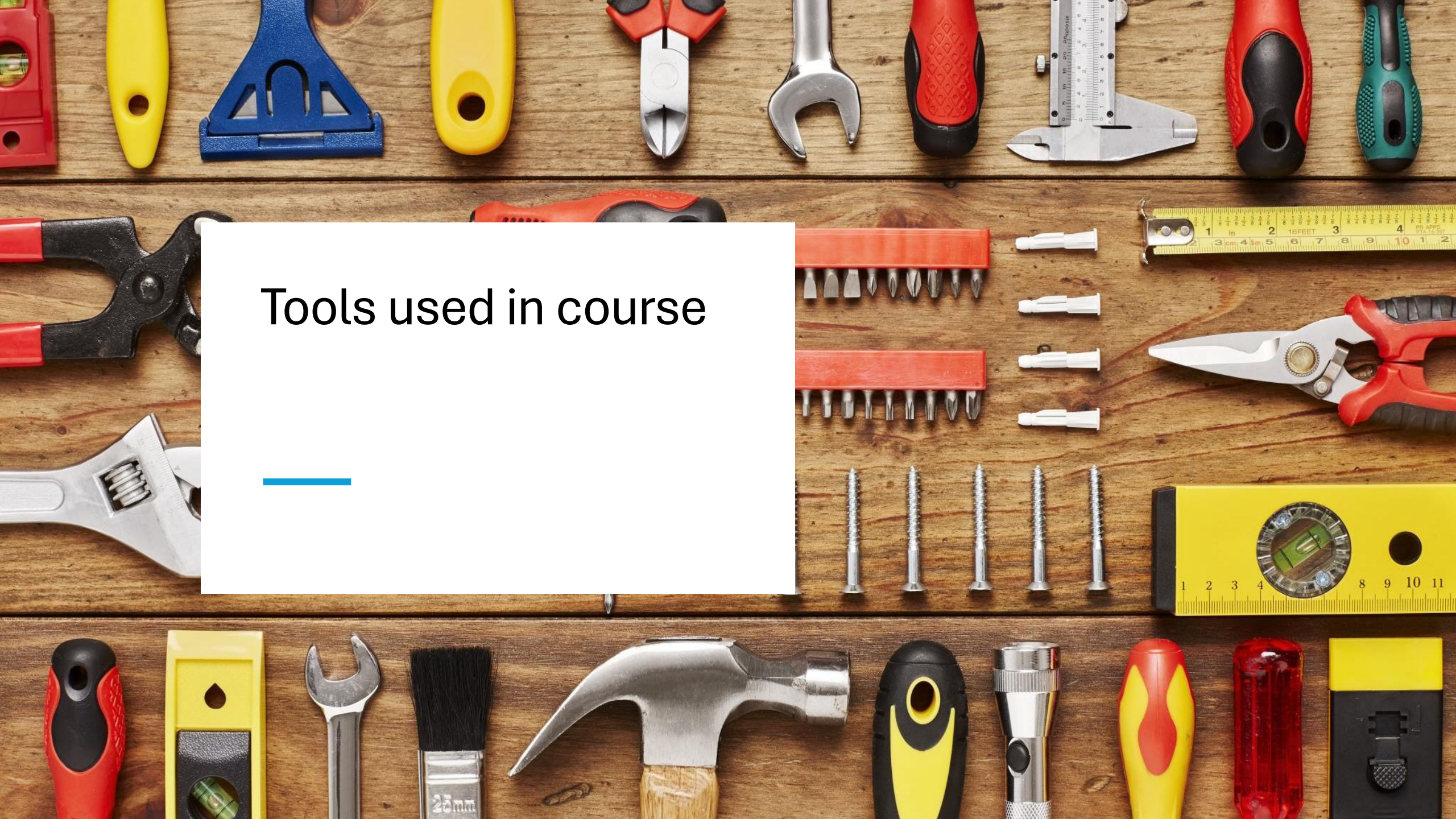
Task 1: Send an email to David requesting a project update.
Task 2: Send a meeting invite to David for next Friday at 15:00.

Select the required tools and plan/execute the tasks accordingly





Tools Setup



Tools used in course

Tools/Solutions

n8n

- Workflow automation tools
- Huge integration ecosystem
- Great for event-driven, multi-app workflows
- Can combine AI with pure automation seamlessly
- Self-host or cloud options

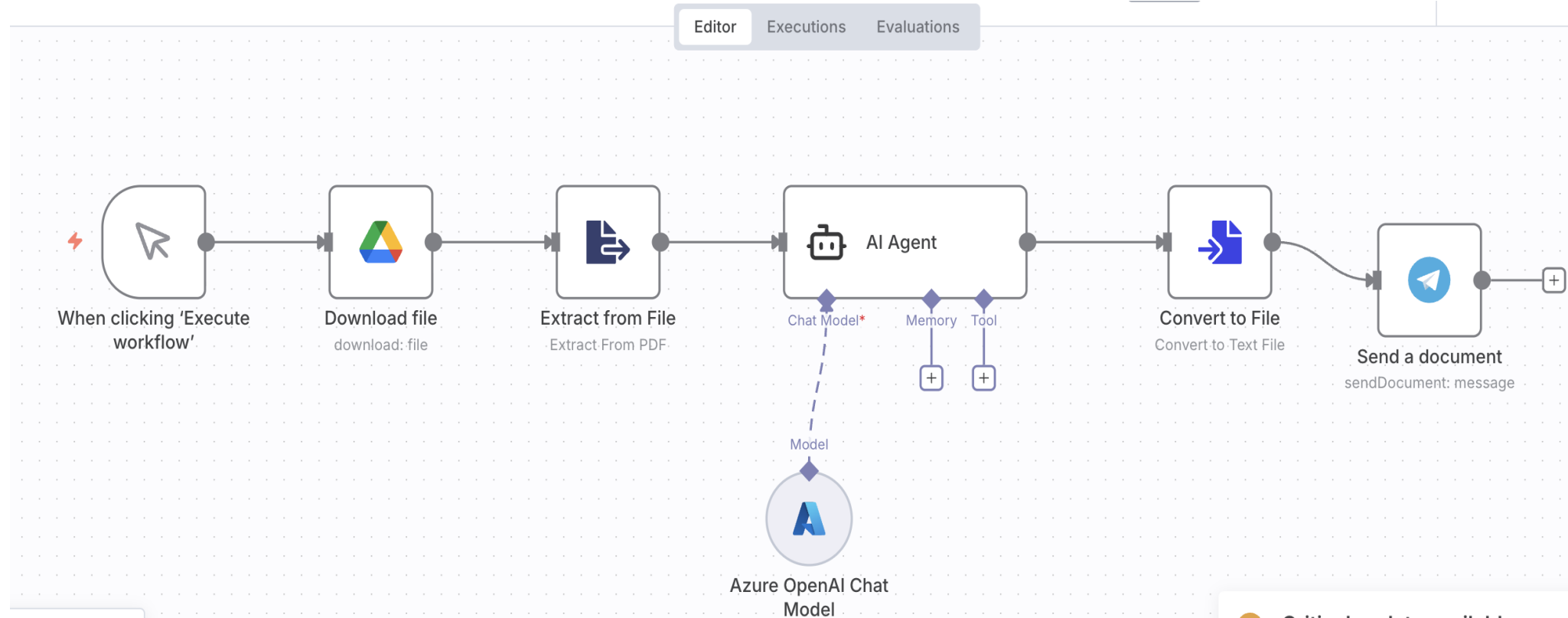
Dify

- AI-native application development platform (LLMOps + Backend-as-a-Service for generative AI)
- Building, deploying, and managing LLM-based apps: chatbots, agents, RAG pipelines, prompt orchestration, knowledge bases.

Python

- Top candidate in programming language for task automation and AI project delivery

n8n Workflow (Example)



n8n-Node Categories

Triggers

- Where, when and how of the agentic workflow begins

Actions

- Perform tasks in an app (e.g. upload file in Google drive)

Utility

- Transform data

Code

- Run code, send HTTP requests and work on webhooks etc

Advanced AI

- Langchain and LLM integration

Overall setup

The screenshot shows the Dify.ai website in a web browser. The browser's address bar shows 'dify.ai'. The website's header includes the Dify logo, navigation links (Marketplace, Solutions, Pricing, Docs, Blog, Forum), and a 'NEW' badge. A 'Get Started' button is highlighted with a red box, and an arrow points to it with the text 'Login here'. The main content area features the headline 'Build Production Ready Agentic Workflow' and a sub-headline '118.5k stars on GitHub'. A 'Get Started' button is also present in the main content area. The footer displays logos for ThermoFisher Scientific, Panasonic, GLENCORE, 價格.com, NTT, and RICOH.

Forum Now Live Join discussions, share insights, and explore how others are building with Dify. ↗

Forum Now Live Join discussions, share insights, and explore how others are building with Dify. ↗

Forum Now Live Join discussions, share insights, and explore how others are building with Dify. ↗

Dify Marketplace Solutions Pricing Docs Blog Forum **NEW** 118.5k **Get Started** → EN

Forum Now Live Join discussions, share insights, and explore how others are building with Dify. ↗

Build Production Ready Agentic Workflow

• 118.5k stars on [GitHub](#)

Dify offers everything you need — agentic workflows, RAG pipelines, integrations, and observability — all in one place, putting AI power into your hands.

Get Started →

ThermoFisher SCIENTIFIC **Panasonic** GLENCORE 價格.com **NTT** **RICOH**

Overall setup

Sample Templates Develop solution workflow here Adding Tools In the workflow Setup LLM models Or data sources

The screenshot shows the Dify AI web application interface. At the top, a navigation bar contains the Dify logo, a dropdown menu for 'Dify's Workspace', an 'Upgrade' button, and a set of icons for 'Explore', 'Studio', 'Knowledge', and 'Tools'. A red box highlights these four icons. To the right of this bar is a 'Plugins' button, also highlighted with a red box. Below the navigation bar, there are tabs for 'All', 'Workflow', 'Chatflow', 'Chatbot', 'Agent', and 'Completion'. The main content area displays a grid of application cards. The first card is 'CREATE APP' with options 'Create from Blank', 'Create from Template', and 'Import DSL file'. The other cards are 'agent_test1', 'RAG_test1', 'Knowledge Retrieval + Chatbot', and 'Simple Document Summarization'. A red box highlights the 'Knowledge' icon in the navigation bar and the 'RAG_test1' card in the main content area. Arrows point from the text labels at the top to these specific elements: 'Sample Templates' points to the 'Explore' icon, 'Develop solution workflow here' points to the 'Studio' icon, 'Adding Tools In the workflow' points to the 'Tools' icon, and 'Setup LLM models Or data sources' points to the 'Plugins' button. Another arrow points from 'Create knowledge base (RAG)' to the 'RAG_test1' card.

cloud.dify.ai/apps

Finish update

Explore Studio Knowledge Tools

Plugins

CREATE APP

- Create from Blank
- Create from Template
- Import DSL file

agent_test1

RAG_test1

Knowledge Retrieval + Chatbot

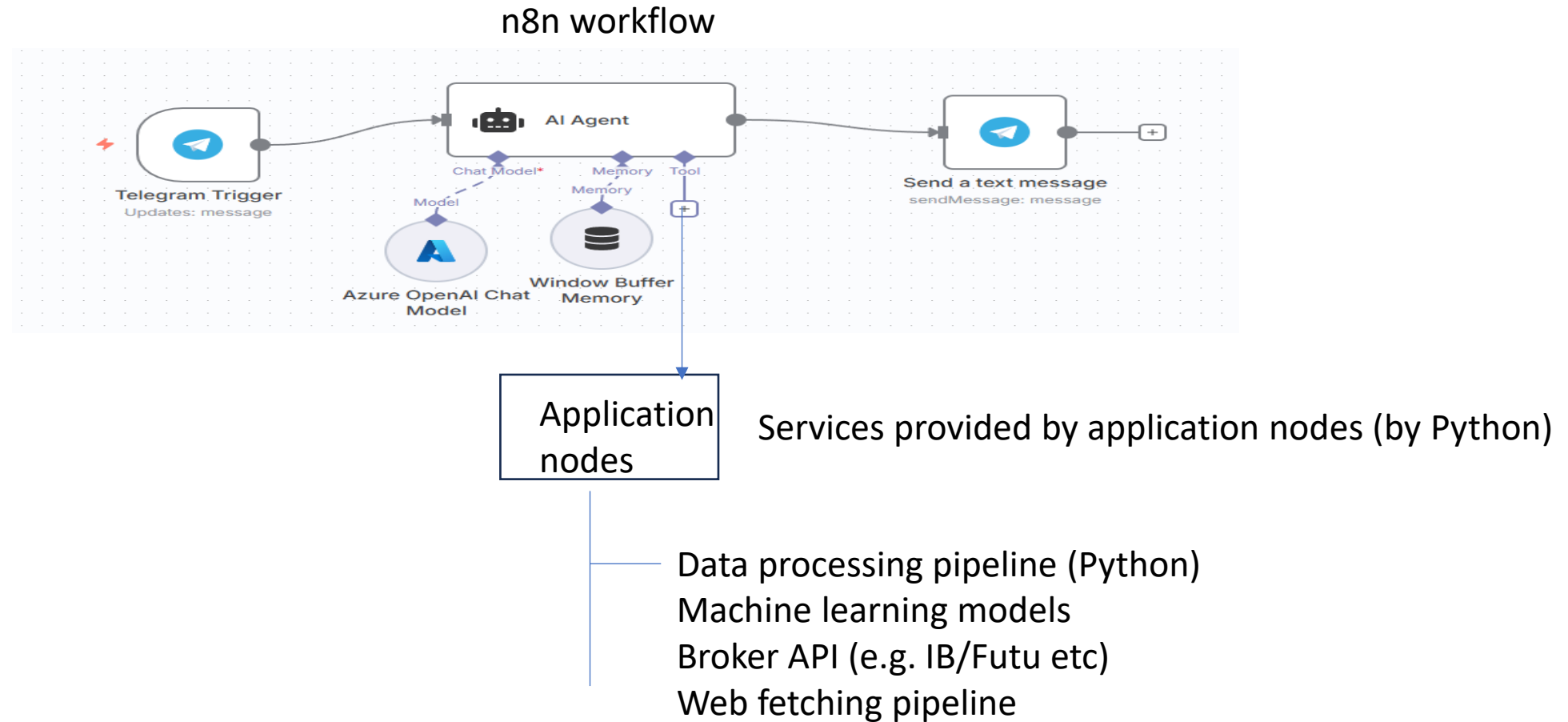
Simple Document Summarization

Create knowledge base (RAG)

Why we use Python

- Open source
- Easy to use
- Top 2 popular programming languages in the world
- Lots of users and developers with big user community
- Lots of useful packages
 - Numpy
 - Pandas
 - Matplotlib
 - Tensorflow
 - Beatifulsoup
 - Selenium
 - Scikit learn
 - And more...
- Most of the popular brokers have Python APIs nowadays
- Good support on data analytics tasks (use of different ML algorithms)

n8n+Python Integration



Why we use Python

- Open source
- Easy to use
- Top 2 popular programming languages in the world
- Lots of users and developers with big user community
- Lots of useful packages
 - Numpy
 - Pandas
 - Matplotlib
 - Tensorflow
 - Beatifulsoup
 - Selenium
 - Scikit learn
 - And more...
- Most of the popular brokers have Python APIs nowadays
- Good support on data analytics tasks (use of different ML algorithms)

Python

Download Python

- <https://www.python.org/downloads/>
- IDE for Python
- Default: IDLE
- Others: Pycharm
<https://www.jetbrains.com/pycharm/download/#section=windows>
- Jupyter notebook
<https://www.anaconda.com/download/>

Use of Jupyter Notebook Data Analysis

- Data Scientist normally uses
- Jupyter notebook for data analysis
 - Statistical analysis
 - Data visualization

Example of using decision tree and random forest

Import data

```
[1]: import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import tree
from plotnine import *
%matplotlib inline
```

```
!pip install pandas plotnine
```

```
[2]: loc = "http://archive.ics.uci.edu/ml/machine-learning-databases/"
ds = "breast-cancer-wisconsin/breast-cancer-wisconsin.data"
url = loc+ds
data=pd.read_csv(url,sep=" ",header=None)
```

```
[3]: data.columns = ["ID", "clumpThickness", "sizeUniformity",
                    "shapeUniformity", "maginalAdhesion",
                    "singleEpithelialCellSize", "bareNuclei",
                    "blandChromatin", "normalNucleoli", "mitosis", "class"]
```

```
[4]: data.head()
```

	ID	clumpThickness	sizeUniformity	shapeUniformity	maginalAdhesion	singleEpithelialCellSize	bareNuclei	blandChromatin	normalNucleoli	mitosis	class
0	1000025	5	1	1	1	2	1	3	1	1	2
1	1002945	5	4	4	5	7	10	3	2	1	2
2	1015425	3	1	1	1	2	2	3	1	1	2
3	1016277	6	8	8	1	3	4	3	7	1	2
4	1017023	4	1	1	3	2	1	3	1	1	2

Task Execution/Automation

While jupyter notebook is used for data analysis, python script will be used for task execution and automation.

The script will be executed with a command line under command prompt such as

>python abc.py

Which means it will use python to execute the python script line by line and from top to bottom.

Sample python script (.py)

```
import pandas as pd
import numpy as np
import yfinance as yf
import time
#import telegram
# from tabulate import tabulate
import matplotlib.pyplot as plt
from datetime import date
import asyncio
#import talib as TA

path="C:/temp/"

# Token and chat ID used in telegram
TOKEN=""
chat_id=""

# Starting date which is used to generate the chart
date1="2021-09-01"
today=date.today().strftime("%d%m%Y")

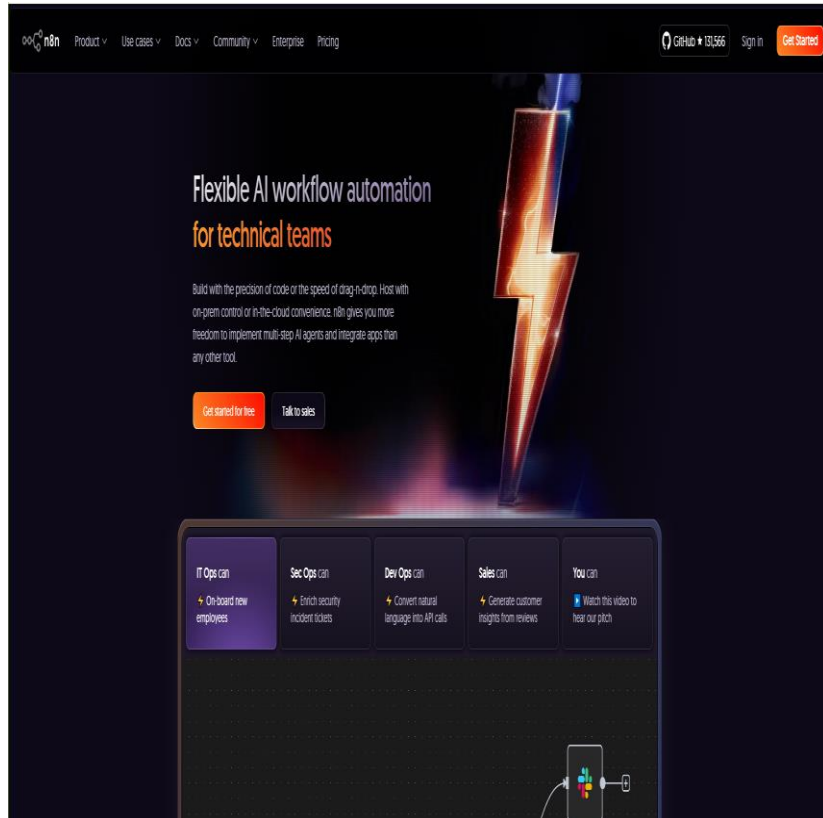
# Import excel with stock codes and create a list L_stock which stores the
# stock codes
L_stocks=pd.read_excel(path+"Stock_monitoring_list2.xlsx")["Stock code"].tolist()
L_stocks.append("HSI")
L_stocks_names=pd.read_excel(path+"Stock_monitoring_list2.xlsx")["Name"].tolist()
L_stocks_names.append("HSI")

# To download the stock data one by one and save it as a dictionary
def getdata(L):
    D1={}
    for i in L:
        df=yf.download(i,"2021-01-01",progress=False)
        message="Now downloading data:"+i
        print(message)
        D1[i]=df
        time.sleep(2)
    return(D1)

# Based on the dictionary above generates the result with the restuen of previous 5 days and
# previous 20 days
def table_gen1(D):
    L_return_1day=[]
    L_return_5days=[]
    L_return_20days=[]
    for i in L_stocks:
        return1day=round((100*(D[i]["Close"].iloc[-1]-D[i]["Close"].iloc[-2])/D[i]["Close"].iloc[-1],1)
        return5days=round((100*(D[i]["Close"].iloc[-1]-D[i]["Close"].iloc[-5])/D[i]["Close"].iloc[-1],1)
        return20days=round((100*(D[i]["Close"].iloc[-1]-D[i]["Close"].iloc[-20])/D[i]["Close"].iloc[-1],1)
        L_return_1day.append(return1day)
        L_return_5days.append(return5days)
```

N8N

n8n.io



- There are two modes of working in n8n
- Cloud based
- Self-host

Installation of n8n (local installation)

Please follow the installation procedure (attached word document)

Types of nodes in n8n-1/2

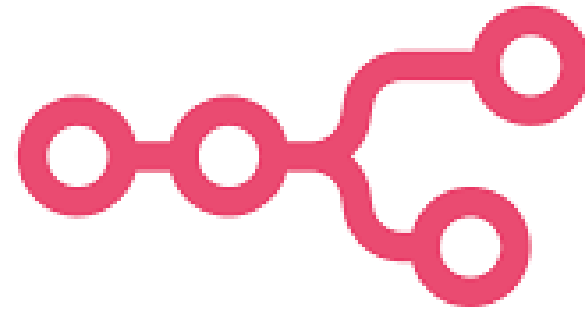
Category	Purpose	Examples of Nodes
Trigger Nodes	Start a workflow when something happens	Webhook Trigger, Schedule Trigger (Cron), Gmail Trigger (new email), Telegram Trigger (new message)
App / Action Nodes	Perform operations with specific apps & services	Google Sheets (add row), Gmail (send email), Slack (post message), Notion (create page), Airtable (update record)
Data Transformation	Clean, reshape, or restructure data	Set, Move/Remove Fields, Rename Keys, Item Lists, Split In Batches
Logic / Control Flow	Control workflow execution path	IF, Switch, Merge, Wait, Error Trigger
Function / Code	Add custom programming logic	Function (JavaScript), Function Item (per item JS), Code (JavaScript / Python)
AI & LangChain	AI reasoning & tool calling	OpenAI, Azure OpenAI, AI Agent (LangChain), Think Node, Embeddings
Utility Nodes	General-purpose helpers	HTTP Request, Webhook, Date & Time, Crypto, Spreadsheet File, File Operations
Database Nodes	Read/write structured data	MySQL, PostgreSQL, MongoDB, SQLite, Notion DB, Airtable
Messaging & Notifications	Communicate with users	Email (IMAP/SMTP/Gmail/Outlook), Slack, Teams, Telegram, Discord, Twilio (SMS/WhatsApp)

Types of nodes in n8n-2/2

Node	
Read/Write files from Disk	Import files(e.g. pdf/csv) to/from binary
Extract from file	Convert binary file to JSON
Convert to file	Convert JSON data to binary file
Edit Fields	Create/edit content (JSON)
AI Agent	AI framework
Memory	Memory module used in AI framework
LLM	e.g. Azure and other LLM
Web Hook	To receive http requests from other server
Command Node	To execute command (e.g. execute bat file under Windows OS)
HTTP Request	To send HTTP request to other nodes/server/AI Agent
Wait	To pend and wait for further message (e.g. web hook) or condition (time expired)
IF/SWITCH	Condition branching
Application nodes	Google/Meta/Telegram/Whatsapp/etc



n8n vs different agent platform

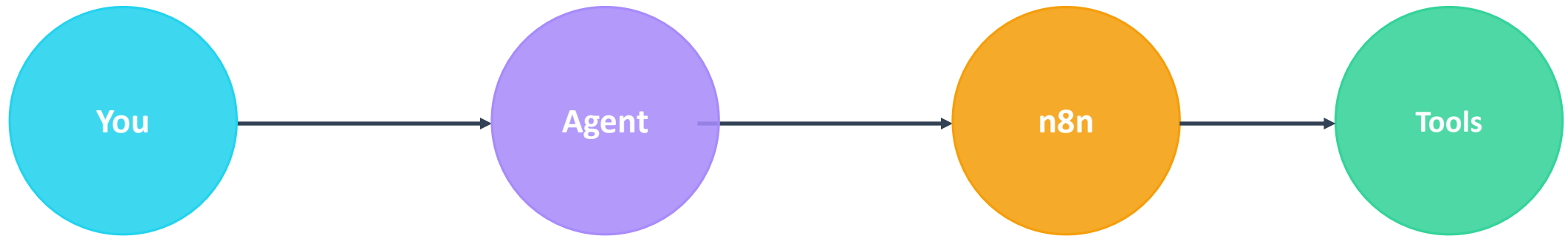


n8n



Agent Platforms vs n8n

What are OpenClaw / Hermes-style agents, and when do they add value beyond n8n + LLM tool calling?



The core message

Agents are not “better n8n”. They solve a different layer.

n8n + LLM already handles natural language → tool calling.

OpenClaw / Hermes become useful when the assistant itself needs to be persistent, multi-channel, memory-aware, or self-improving.

**n8n
workflow engine**

**LLM tool calling
reasoning step**

**Agent runtime
assistant layer**

Use the agent layer only when the user experience and autonomy model need it.

Three things people often confuse

Same words, different architectural responsibilities.

n8n

Visible workflow automation

- Triggers, nodes, branches
- APIs, credentials, retries
- Great for deterministic jobs with LLM support

OpenClaw

Gateway-first assistant platform

- One gateway across chat apps
- Sessions, channels, routing
- Assistant “from your pocket”

Hermes Agent

Agent-first self-improving runtime

- Creates / improves skills
- Searches past conversations
- Learns across sessions

Architecture A: n8n + LLM only

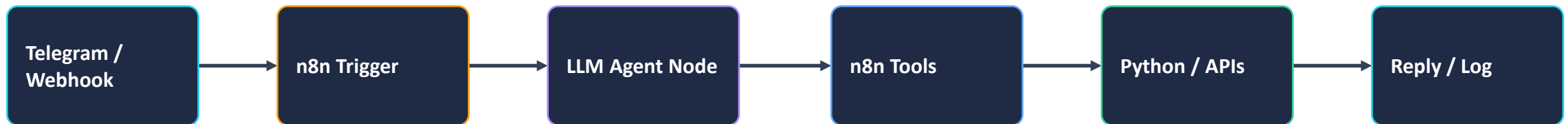
Best when the process is clear and can be modeled as a workflow.

Good fit

- One or two chat channels
- Clear commands: /check, /backtest, /risk
- Deterministic trading workflows
- Easier debugging and lower complexity

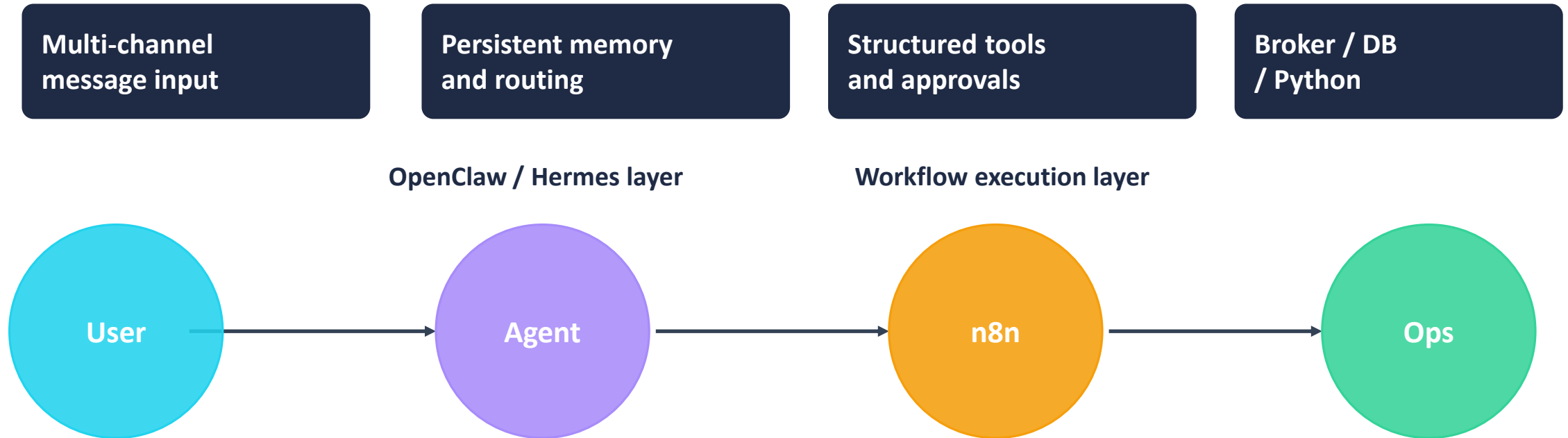
Limit

- You must build session memory, routing, identity, and multi-channel behavior yourself
- The assistant experience is tied to each workflow



Architecture B: Agent platform + n8n

Best when you want an always-available assistant, not just a workflow.



The agent platform becomes the front desk. n8n stays the operations engine.

Where the difference really shows up

Not in “understanding English”; the LLM already does that.

Question	n8n + LLM only	OpenClaw / Hermes + n8n
Where does conversation live?	Inside each workflow or chat trigger	In a persistent assistant runtime
How many channels?	Each channel needs wiring	Gateway-style multi-channel entry
How does it learn?	Custom memory database needed	Hermes emphasizes built-in learning loop
How are workflows run?	Native n8n execution	Agent calls n8n as tools/webhooks
Best for	Reliable repeatable automation	Ambiguous requests and ongoing assistant work

When the agent layer is justified

Add OpenClaw / Hermes only when one of these becomes a real requirement.

1

Multi-channel assistant

Same assistant across Telegram, WhatsApp, Slack, desktop, etc.

2

Persistent sessions

Ongoing context: “continue the analysis from yesterday.”

3

Self-improving skills

Hermes-style learning loop: create/improve skills from experience.

4

Delegated fuzzy work

Research, debugging, summarizing, planning before calling workflows.

5

Assistant identity

Separate personas, workspaces, trust boundaries, and tool policies.

**When the user wants to treat the system like a real trading partner/assistant rather than a tool.
Otherwise: keep it simple with n8n + LLM.**

Algo trading cases: which layer owns what?

Use agents for interaction and judgment; use n8n/Python for repeatable execution.

Market scan

Ask: "Check NQ / QQQ today"

n8n runs data + indicators + report

Backtest

Natural-language parameters

Python service calculates metrics + charts

Options scanner

Clarify risk profile / expiry

n8n filters option chain + ranks spreads

Risk monitor

Explain portfolio exposure

n8n pulls broker positions + limits

Trade journal

Q&A on performance

n8n imports trades + updates database

Agent layer

n8n / Python layer

Decision matrix

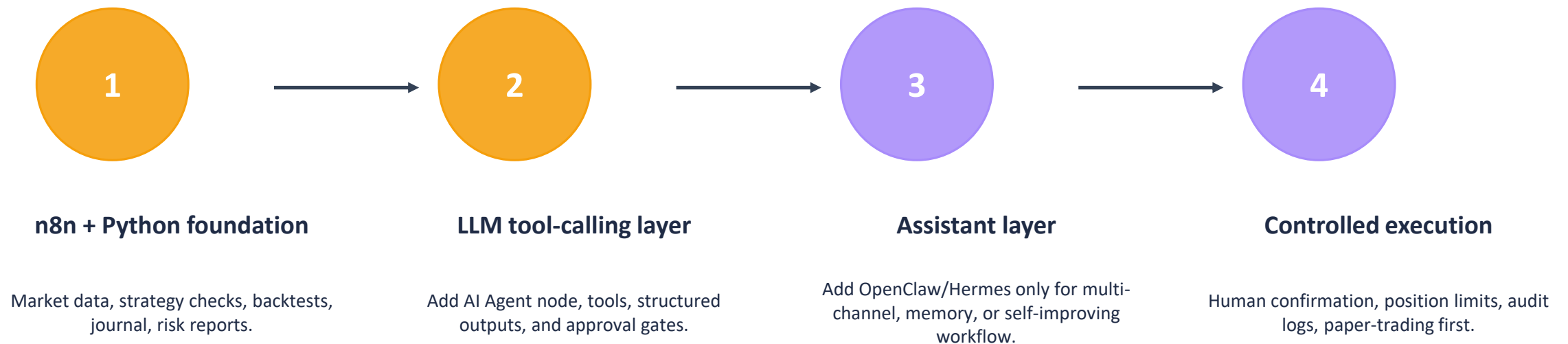
The simplest way to decide whether OpenClaw / Hermes adds value.

Need / condition	n8n + LLM	Add OpenClaw / Hermes
Single Telegram trading bot	Yes	No
Structured scheduled market scans	Yes	No
Natural-language tool calling	Yes	No
Multi-channel assistant	Possible	Yes
Long-lived memory / persona	Custom build	Yes
Self-improving skills	Custom build	Hermes fit
Operations reliability / audit logs	Yes	Use n8n anyway
Live order execution	Only with controls	Agent should not directly own execution

Recommendation: start simple; add the agent platform only after the workflow layer is valuable.

Recommended roadmap for your trading assistant

Build value before adding an assistant gateway.



Do not start with OpenClaw/Hermes just because it is new. Start with workflows that produce trading value.

Trading safety: the red line

Agent autonomy is powerful, but trading systems need guardrails.

Allowed automatically

- Read market data
- Run backtests
- Summarize positions
- Generate reports
- Update journal

Require human approval

- Send email / message
- Change strategy config
- Modify risk parameters
- Pause/resume bot

Strongly restricted

- Place live order
- Cancel or modify order
- Close position
- Increase size or leverage

For live trading, the broker execution service should enforce limits even if the agent makes a bad request.

Final positioning

How to explain the difference clearly in one sentence.

**n8n is the execution map.
OpenClaw/Hermes are the assistant runtime.
The LLM is the reasoning engine.**

For algo trading: keep strategy, risk, and execution deterministic; use agents for conversation, research, explanation, and controlled delegation.

Best first build: Telegram → n8n → LLM Agent → Python trading tools → Report / Approval

Local n8n login account

User name: aiagent@fevaworks.com

Password: Feva3106